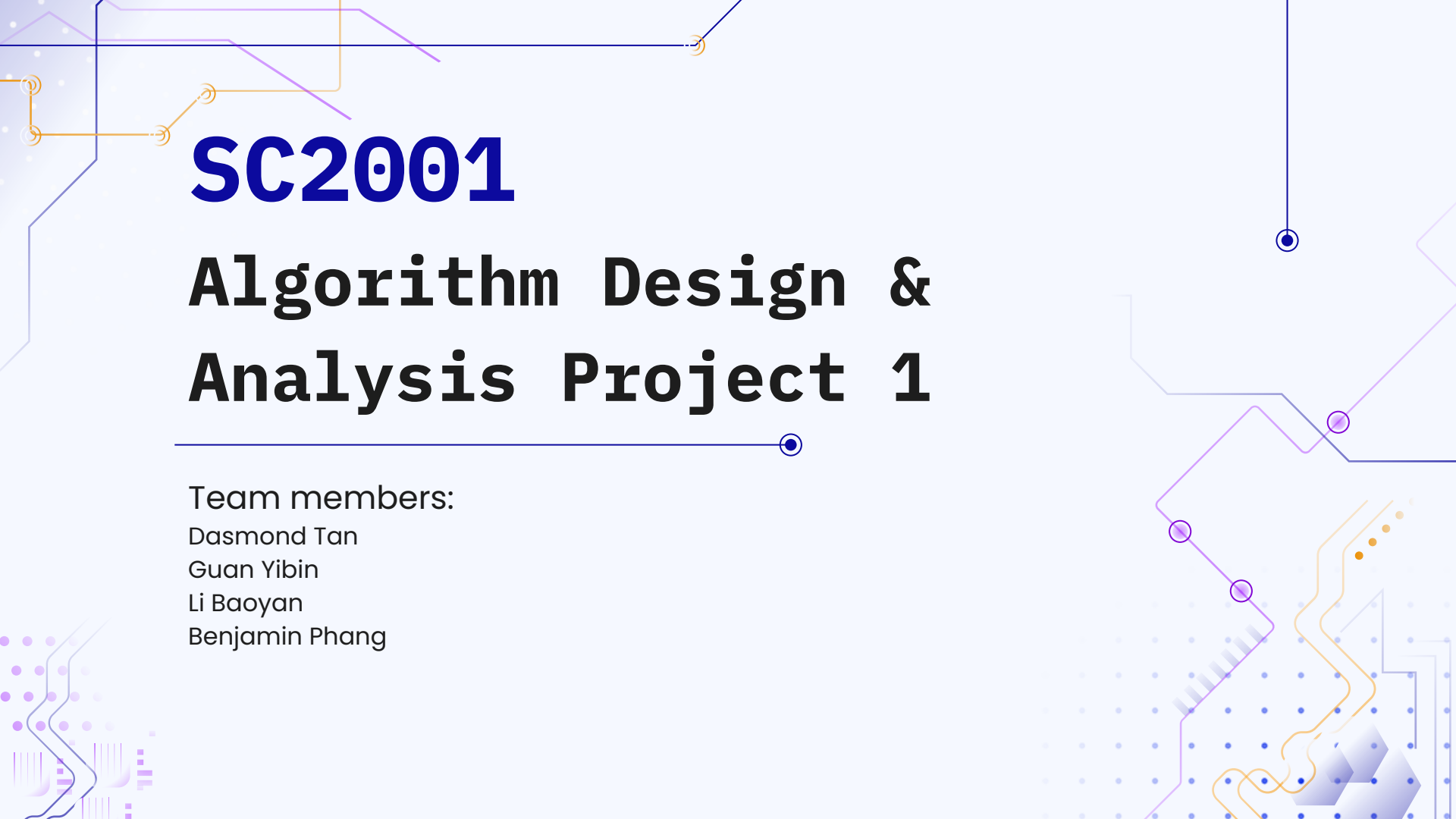




SC2001

Algorithm Design & Analysis Project 1

Team members:

Dasmond Tan

Guan Yibin

Li Baoyan

Benjamin Phang

1) Implementation of Hybrid Sort

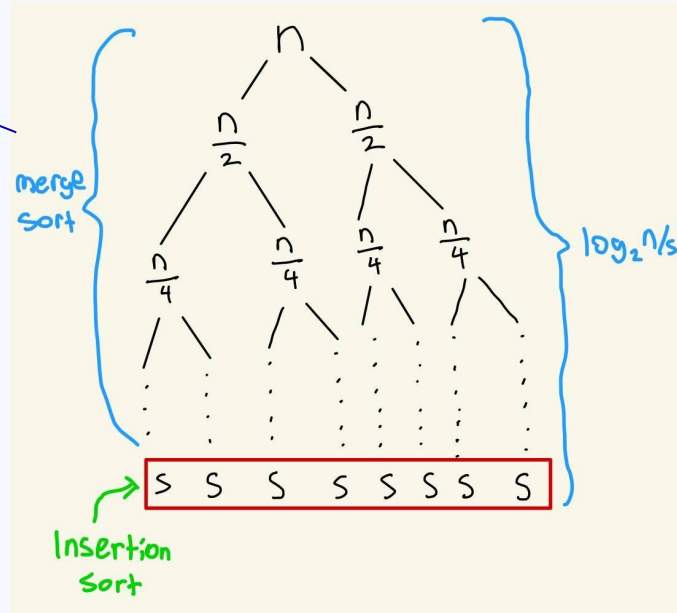
Merge sort is great for large array
- $O(n \log n)$ time

But...

Recursive splitting adds overhead from many small function calls.

So... What if we stop recursion early?

Use a simpler algorithm like **Insertion Sort** for very small arrays



Hybrid Function

```
/* Hybrid sort: insertion when run size <= S */
static void hybrid_sort_rec(int *a, int *aux, int left, int right, int S) {
    int len = right - left + 1;
    if (len <= 1) return;
    if (len <= S) {
        insertion_sort(a, left, right);
        return;
    }
    int mid = left + (right - left) / 2;
    hybrid_sort_rec(a, aux, left, mid, S);
    hybrid_sort_rec(a, aux, mid + 1, right, S);
    merge(a, aux, left, mid, right);
}
```

Threshold **S**

Determine the type of sorting

If the problem size is small ($\text{len} \leq S$)

- insertion_sort

Else

- Continue the recursive splitting

} Recursive Splitting

Insertion Sort (for Small arrays)

```
/* Insertion sort on subarray [left, right] inclusive */
static void insertion_sort(int *a, int left, int right) {
    for (int i = left + 1; i <= right; ++i) {
        int key = a[i];
        int j = i - 1;
        while (j >= left) {
            g_comparisons++;           /* compare a[j] > key */
            if (a[j] > key) {
                a[j + 1] = a[j];
                --j;
            } else {
                break;
            }
        }
        a[j + 1] = key;
    }
}
```

Perfect tool for sorting final, small subarrays of size S

Merge Sort (for Large arrays)

```
static void merge(int *a, int *aux, int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;

    /* copy to aux */
    for (int t = left; t <= right; ++t) aux[t] = a[t];

    while (i <= mid && j <= right) {
        g_comparisons++;           /* compare aux[i] <= aux[j] */
        if (aux[i] <= aux[j]) a[k++] = aux[i++];
        else a[k++] = aux[j++];
    }
    while (i <= mid) a[k++] = aux[i++];
    while (j <= right) a[k++] = aux[j++];
}
```

Extremely efficient at combining large, sorted lists ($O(n)$ time)

Constant Variables Declared

sizes of arrays

```
static const int SIZES_VAR_N[] = {1000, 2000, 4000, 8000, 10000, 15000, 25000, 50000, 100000, 150000, 200000, 250000,
                                   500000, 750000, 1000000, 2000000, 4000000, 8000000, 10000000};
//static const int SIZES_VAR_N[] = {1000, 2000, 4000, 8000, 10000, 15000, 25000, 50000, 100000, 150000, 200000, 250000, \
                                   500000, 750000, 1000000, 2000000, 4000000, 8000000, 10000000};
static const int SIZES_VAR_N_COUNT = sizeof(SIZES_VAR_N) / sizeof(SIZES_VAR_N[0]);

/* (c.iii) Candidate S values when searching for optimal S per n */
static const int S_CANDIDATES[] = {0, 2, 4, 8, 16, 32, 64}; //, 128}; //removing 1
// static const int S_CANDIDATES[] = {0, 4, 8, 12, 16, 18, 20, 21, 22, 23, 24, 26, 28, 32, 40, 48, 64};
static const int S_CANDIDATES_COUNT = sizeof(S_CANDIDATES) / sizeof(S_CANDIDATES[0]);

/* trials per configuration (average variability) */
static const int TRIALS = 3;           // number of trials to run sorts on same data
static const int SAMPLING_TRIALS = 30; // number of samplings of means
/* Largest random value (x) for [1..x] */
static const int MAX_RAND_VALUE = 10000000;
static const int micros_sec = 0; // true false expression to convert to micro seconds , true yes false no
```

varying S values

Our approach

For array_length in list_of_array_length:

 Create array of size array_length

 For s in list_of_S:

 For second_run in range(second_run_count):

 For first_run in range(first_run_count):

 Run the hybrid_sort with s as threshold

 Accumulate data for each variable in each first_run

 Divide data obtained in first_run loop by first_run_count

 Accumulate divided data for each variable obtained across number of second_run_count

 Divide data in second_run loop by second_run_count

 Output data obtained to csv for every s in every array

2) Generate Input Data

Generate arrays of increasing sizes, in a range from 1,000 to 10 million

```
static int *alloc_array(int n) {  
    int *p = (int*)malloc((size_t)n * sizeof(int));  
    if (!p) {  
        fprintf(stderr, "Allocation failed for %d ints\n", n);  
        exit(1);  
    }  
    return p;  
}
```

Allocate memory for
an array of size n

```
static const int SIZES_VAR_N[] = {  
    1000, 2000, 4000, 8000, 10000, 15000, 25000, 50000,  
    100000, 150000, 200000, 250000, 500000, 750000,  
    1000000, 2000000, 4000000, 8000000, 10000000  
};  
static const int SIZES_VAR_N_COUNT = sizeof(SIZES_VAR_N)/sizeof(SIZES_VAR_N[0]);
```

Generate data

Fill up the array with random number range from [1,x]

```
static void fill_random(int *a, int n, int maxv) {  
    for (int i = 0; i < n; ++i) a[i] = rand_1_to_max(maxv);  
}
```

Generated Samples

Array size = 1000,	Min = 26143,	Max = 9994599
Array size = 2000,	Min = 8202,	Max = 9999333
Array size = 4000,	Min = 5729,	Max = 9999554
Array size = 8000,	Min = 701,	Max = 9998535
Array size = 10000,	Min = 366,	Max = 9999525
Array size = 15000,	Min = 351,	Max = 9997373
Array size = 25000,	Min = 370,	Max = 9999722
Array size = 50000,	Min = 321,	Max = 9999969
Array size = 100000,	Min = 89,	Max = 9999909
Array size = 150000,	Min = 98,	Max = 9999991
Array size = 200000,	Min = 10,	Max = 9999917
Array size = 250000,	Min = 68,	Max = 9999945
Array size = 500000,	Min = 12,	Max = 9999978
Array size = 750000,	Min = 9,	Max = 9999991
Array size = 1000000,	Min = 12,	Max = 9999989
Array size = 2000000,	Min = 6,	Max = 9999991
Array size = 4000000,	Min = 4,	Max = 9999993
Array size = 8000000,	Min = 1,	Max = 10000000
Array size = 10000000,	Min = 1,	Max = 9999999

2) Analysis of Time Complexity

Theoretically:

Insertion Sort Algorithm

Best Case: $O(s)$
Worst Case: $O(s^2)$

Merge Sort Algorithm

Best Case: $O(n \log n)$
Worst Case: $O(n \log n)$

Implementing
Hybrid Algorithm
(n/s)

$O(n/s^2) = O(n)$
Best Case: $O(n)$
Worst Case: $O(ns)$
 $O(n/s * s^2) = O(ns)$

Best Case: $O(n \log n / s)$
Worst Case: $O(n \log n / s)$

$2^i * s = n$
 $2^i = n/s$
 $i * \log(2) = \log(n/s)$
 $i = \log(n/s)$

Total Time Complexity of Hybrid Sort Algorithm
=
Insertion Sort + Merge Sort

Best Case: $O(n + n \log(n/s))$

Worst Case: $O(ns + \log(n/s))$

ns and n/s are
constants!

When S is Fixed

$O(n + n \log(n)) = O(n \log n)$

Plotting Code to Filter Fixed s and n

```
def plot_all(csv_path=None, save_folder=None, group_by=None):
    base = _base_dir()
    csv_path = csv_path or os.path.join(base, CSV_FILE)
    out_dir = save_folder or os.path.join(base, SAVE_FOLDER)
    os.makedirs(out_dir, exist_ok=True)

    df = _load_dataframe(csv_path)
    time_col = _pick_time_col(df)

    # choose grouping
    group_by = (group_by or GROUP_BY)
    if group_by not in ("S", "n"):
        raise ValueError("group_by must be 'S' or 'n'")

    selected_index = group_by
    not_selected_index = "n" if group_by == "S" else "S"

    unique_list = sorted(df[selected_index].dropna().astype(int).unique().tolist())
    print(f"Unique {selected_index} values: {unique_list}")

    for u in unique_list:
        df_fixed = df[df[selected_index] == u].sort_values(by=not_selected_index)
        if df_fixed.empty:
            continue

        # ----- Plot comparisons -----
        x = df_fixed[not_selected_index].to_numpy()
        y = df_fixed[comparisons].to_numpy()

        plt.figure(figsize=(10,6))
        plt.plot(x, y, "o-", label="Empirical comparisons")

        if not_selected_index == "n":
            # x = n -> classic O(n log n)
            if len(df_fixed) >= 2 and np.all(x > 1):
                y_th = _theory_nlogn(x, y)
                plt.plot(x, y_th, "--", label="Theoretical O(n log n)")
        else:
            # x = S -> optional hybrid theory
            if PLOT_HYBRID_THEORY_WHEN_S and len(df_fixed) >= 3:
                n_val = int(df_fixed["n"].iloc[0])
                if HYBRID_THEORY_FIT:
                    y_fit, (a,b) = _hybrid_theory_fitted(n_val, df_fixed["S"].to_numpy(), y)
                    plt.plot(x, y_fit, ":", label=f"Hybrid theory (fitted a={a:.3g}, b={b:.3g})")
                else:
                    y_simpl = _hybrid_theory_simple(n_val, df_fixed["S"].to_numpy())
                    plt.plot(x, y_simpl, ":", label="Hybrid theory (a=1, b=1/4)")
```

----- Config (edit these) -----

#CSV_FILE = "results.csv" # input CSV

CSV_FILE = "results.csv"

SAVE_FOLDER = "adjusted plots"

where to save images

FILTER_ALGO = None

e.g. "hybrid" or "mergesort" or None to keep all

DROP_S_EQ_0 = False

set True to hide S=0 rows

GROUP_BY = "n"

"S" -> plot vs n (theory $O(n \log n)$); "n" -> plot vs S

PLOT_HYBRID_THEORY_WHEN_S = True

if GROUP_BY == "n": overlay hybrid theory vs S

HYBRID_THEORY_FIT = True

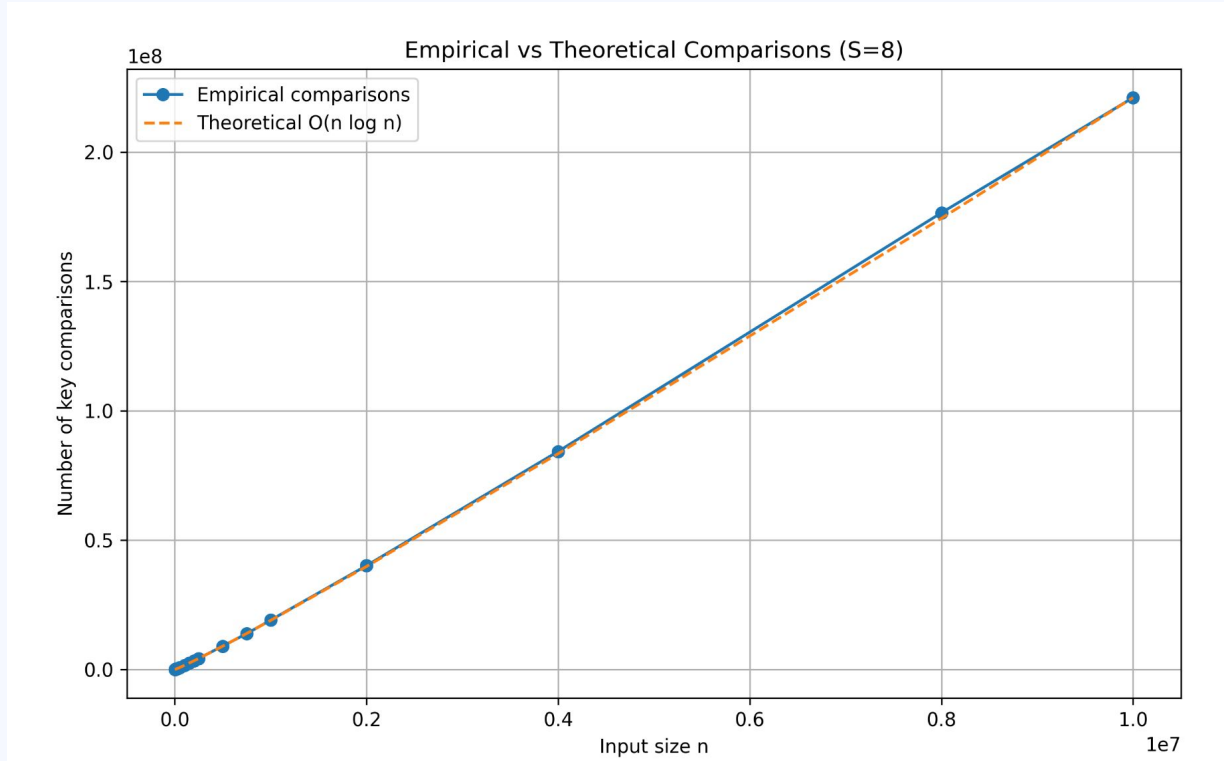
fit constants a,b to data; else use a=1,b=1/4

..

3) i. Analysis of Time Complexity

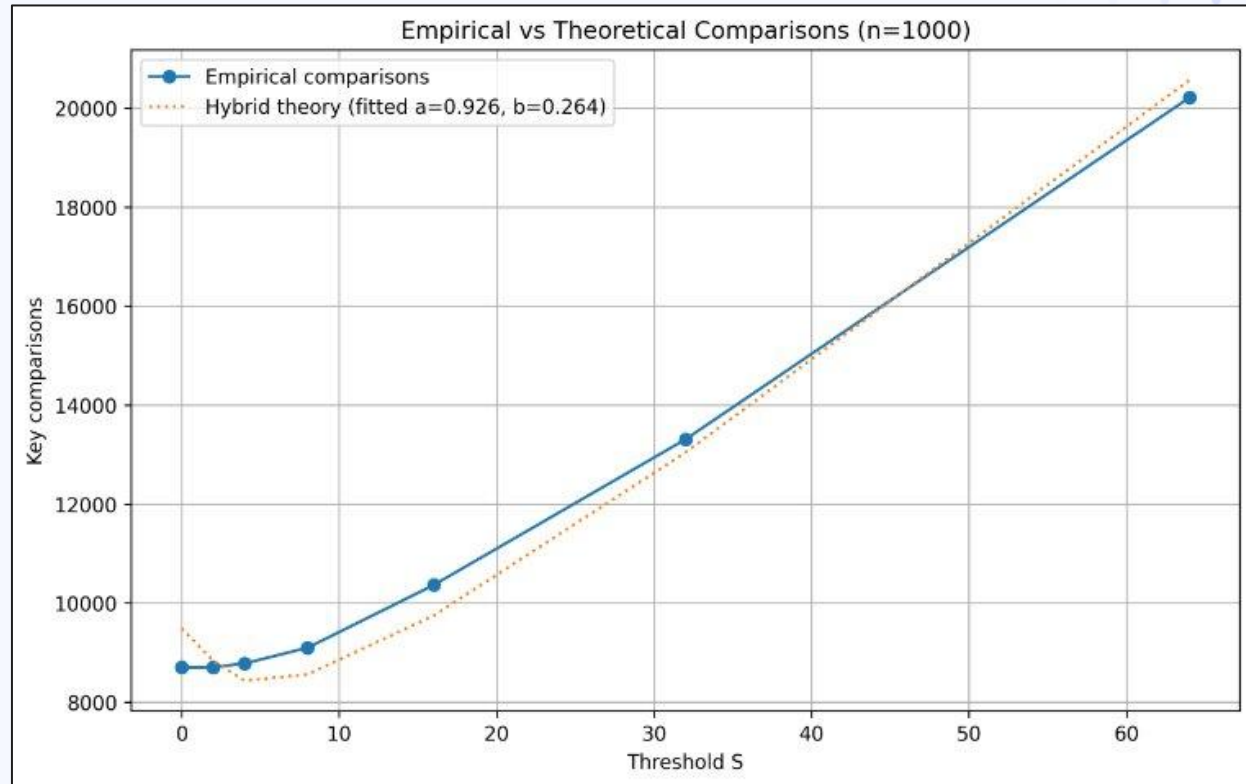
S is **fixed**

similar asymptotic
time complexity as
merge sort



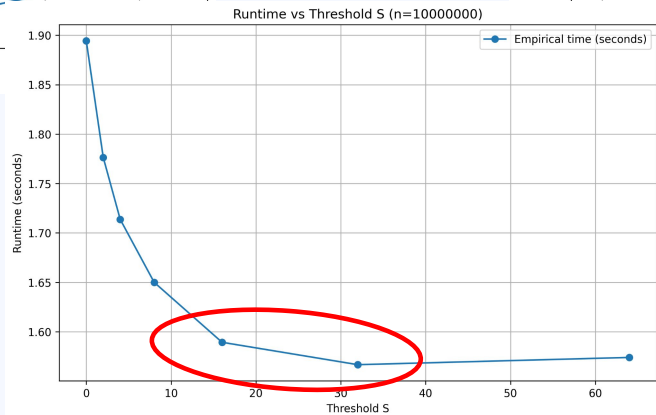
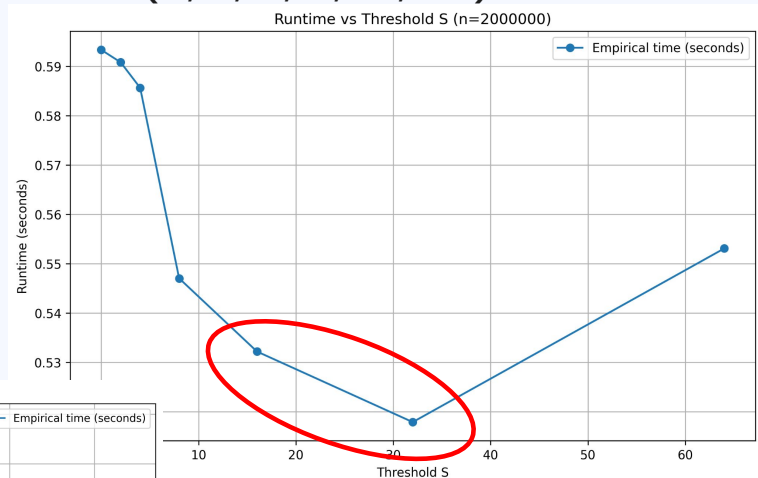
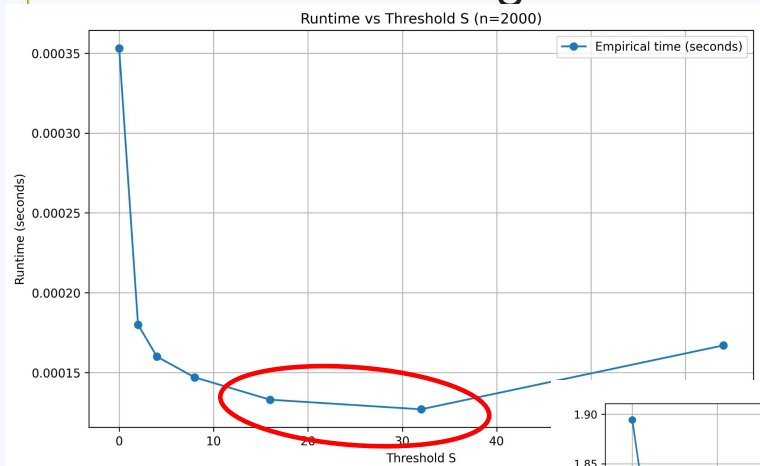
3) ii. Analysis of Time Complexity

n is **fixed**

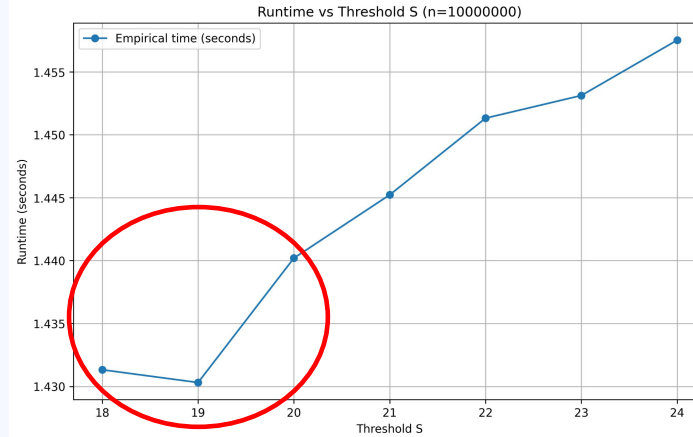
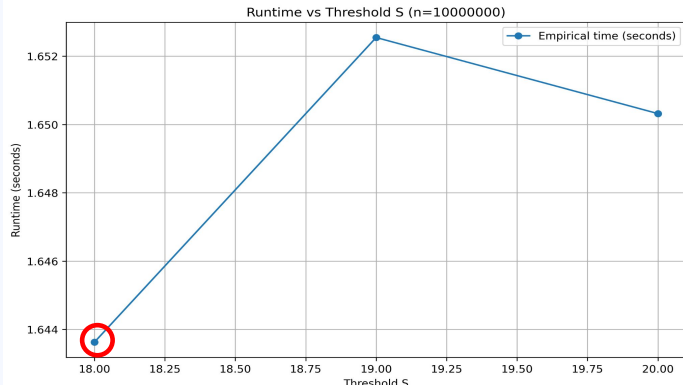
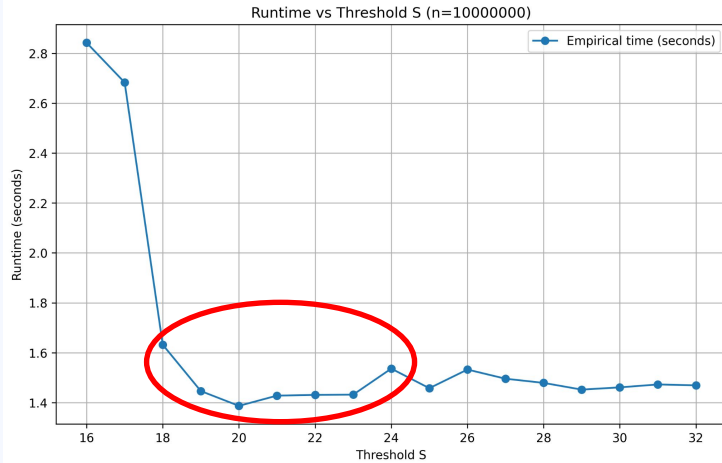


3) iii. Analysis of Time Complexity

Decreasing runtime for starting S values (0, 2, 4, 8, 16, 32)



3) iii. Analysis of Time Complexity

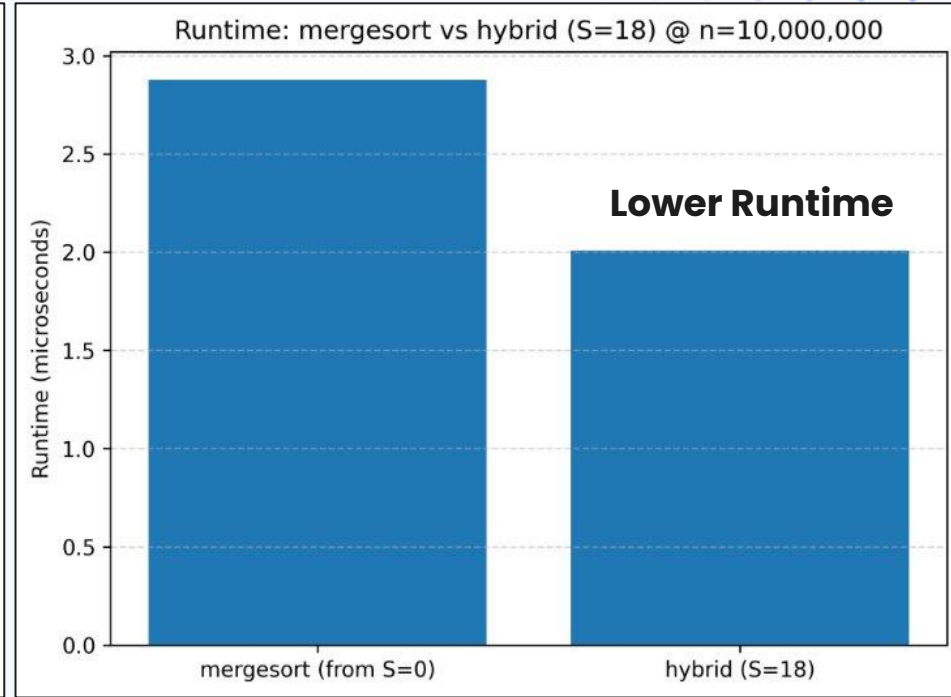
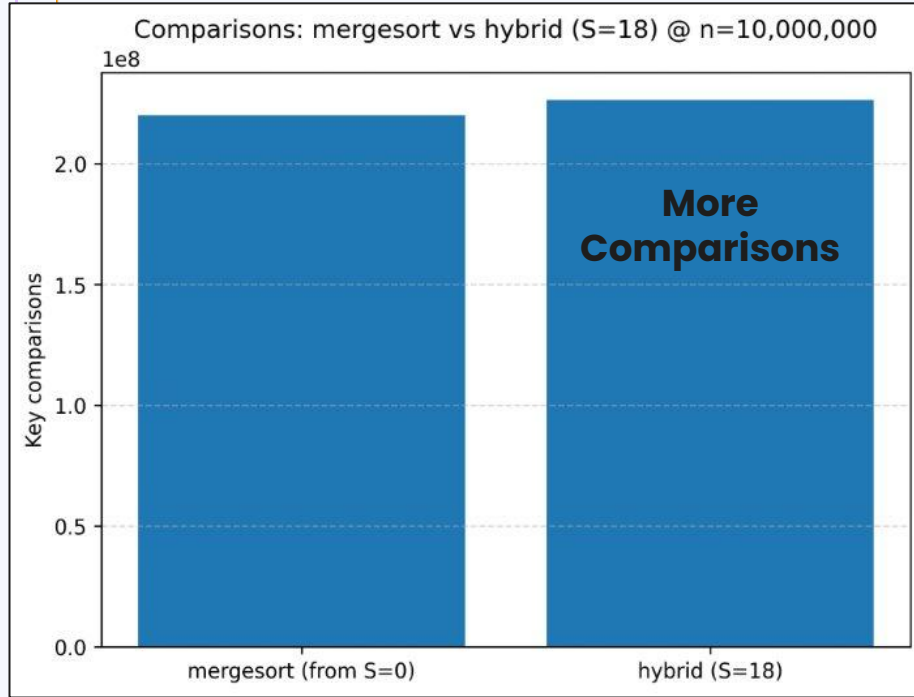


We found that the **optimal value of S is 18**

S = 18 provides a good balance between time efficiency and key comparisons

```
18,0.2295
19,0.3588
20,0.40914285714285715
```

4) Compare with original Mergesort



Conclusion

Hybrid sort with $S = 18$ **outperforms the standard merge sort** by taking advantage of the insertion sort's efficiency on small sub-arrays, reducing recursive calls and memory allocation.

Therefore we can confirm that we have found an optimal S value which provides a notable performance improvement over the standard merge sort.



Thank You!
